

[À COMPLÉTER : Nom de l'école / logo EMSI]

Rapport de Stage / Projet de Fin d'Année (PFA)

**Conception d'une Plateforme de Gestion des
Bénéficiaires
de l'Axe Économie Sociale et Solidaire au sein
de la
Fondation OCP en intégrant des Exigences de
Cybersécurité**

Réalisé par : **[À COMPLÉTER : Nom(s) de l'étudiant(e)]**

Encadrant académique : **[À COMPLÉTER : Nom de l'encadrant
EMSI]**

Encadrant professionnel (Fondation OCP) : **[À COMPLÉTER : Nom
de l'encadrant OCP]**

Année universitaire : **[À COMPLÉTER : 20XX/20XX]**

Fondation OCP — **[À COMPLÉTER : Direction / Axe Économie
Sociale et Solidaire]**

Remerciements

[À COMPLÉTER : Remerciements personnalisés envers l'encadrant académique, l'encadrant professionnel, l'équipe d'accueil, etc.]

Résumé

Ce rapport documente la réalisation de **Secure Requirements Studio (SRS)**, un outil interne d'ingénierie des exigences conçu pour cadrer, spécifier et sécuriser le projet de Plateforme de Gestion des Bénéficiaires de l'Axe Économie Sociale et Solidaire de la Fondation OCP. Le stage s'est concentré sur la conception et le développement de SRS lui-même — un entretien structuré (assisté par IA ou en questionnaire classique) qui transforme les réponses des parties prenantes en un Cahier des Charges, une conception applicative, un diagramme de conception MVP et des exigences de cybersécurité directement traçables aux réponses collectées. **[À COMPLÉTER : Compléter avec les résultats quantitatifs obtenus et les limites précises constatées.]**

Mots-clés : ingénierie des exigences, cybersécurité, Fondation OCP, économie sociale et solidaire, cahier des charges, intelligence artificielle.

Abstract

[À COMPLÉTER : Traduction anglaise du résumé.]

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des matières	iv
Table des figures	v
Liste des tableaux	vi
1 Introduction générale	1
1.1 Objectifs du stage	1
1.2 Structure du rapport	1
2 Présentation de la Fondation OCP	2
2.1 Présentation générale	2
2.2 Mission et domaines d'intervention	2
2.3 Contexte de l'Axe Économie Sociale et Solidaire	2
3 Contexte et problématique	3
3.1 Contexte du projet	3
3.2 Problématique	3
3.3 Besoin métier	3
3.4 Objectifs du projet	3
3.5 Périmètre	4
4 Analyse et conception	5
4.1 Étude de l'existant	5
4.2 Analyse des besoins	5
4.3 Acteurs et rôles	5
4.4 Fonctionnalités principales de SRS	5
4.5 Architecture générale	6
4.6 Conception de la solution	6
4.7 Diagramme de conception du MVP	6
4.8 Flux principaux	6
4.8.1 Flux d'entretien	6
4.8.2 Flux de génération documentaire	7
4.8.3 Flux de revue	7

4.9	Choix technologiques	7
5	Réalisation de Secure Requirements Studio	8
5.1	Environnement de développement	8
5.2	Architecture de Secure Requirements Studio	8
5.3	Questionnaire guidé	8
5.4	Entretien assisté par IA	8
5.5	Intégration des fournisseurs IA	9
5.6	Génération dynamique des exigences	9
5.7	Génération du Cahier des Charges	9
5.8	Génération de la conception	9
5.9	Génération du diagramme MVP	9
5.10	Génération des documents	9
6	Intelligence artificielle et assistance à l'analyse	10
6.1	Objectif de l'assistant IA	10
6.2	Fonctionnement de l'entretien intelligent	10
6.3	Gestion du contexte	10
6.4	Restrictions du domaine de l'IA	10
6.5	Gestion des fournisseurs IA	10
6.6	Fournisseurs IA intégrés	11
6.7	Sécurité des clés API	11
6.8	Limites et risques liés à l'IA	11
7	Cybersécurité	12
7.1	Modèle de menace	12
7.2	Analyse des risques	12
7.3	Contrôle d'accès	12
7.4	Protection des données	12
7.5	Sécurité des fichiers	13
7.6	Journalisation et traçabilité	13
7.7	Sauvegarde et continuité	13
7.8	Secure SDLC	13
8	Tests	14
8.1	Tests fonctionnels	14
8.2	Tests d'intégration	14
8.3	Tests de sécurité	14
8.4	Tests de génération documentaire	14
8.5	Tests de l'assistant IA	15
9	Résultats	16
9.1	Fonctionnalités réalisées	16
9.2	Résultats obtenus	16
9.3	Limites	16
9.4	Perspectives	17
10	Conclusion générale	18

A	Glossaire	19
B	Bibliographie et Webographie	20
C	Guide d'installation	21
C.1	Prérequis	21
C.2	Démarrage rapide	21
C.3	Configuration	21
C.4	Lancement manuel (sans script)	21
C.5	Tests	22
D	Annexes techniques	23
D.1	Arborescence simplifiée du backend	23
D.2	Exemple d'exigence générée	23
D.3	Exemple de diagramme MVP (Mermaid)	24

Table des figures

4.1	Exemple de diagramme de conception MVP généré par SRS	6
-----	---	---

Liste des tableaux

Chapitre 1

Introduction générale

Dans le cadre de [À COMPLÉTER : stage de fin d'année / PFA, nom du diplôme visé, durée exacte du stage], ce rapport présente le travail réalisé au sein de la Fondation OCP autour du projet : « *Conception d'une Plateforme de Gestion des Bénéficiaires de l'Axe Économie Sociale et Solidaire au sein de la Fondation OCP en intégrant des Exigences de Cybersécurité* ».

La Fondation OCP pilote un ensemble de programmes sociaux et économiques regroupés sous l'Axe Économie Sociale et Solidaire, au bénéfice de coopératives et de bénéficiaires directs et indirects à travers le Maroc. Avant de développer une plateforme numérique de gestion pour cet axe, il est indispensable de cadrer précisément les besoins métier, les contraintes techniques et — étant donné la sensibilité des données traitées — les exigences de cybersécurité et de protection des données personnelles.

C'est ce besoin de cadrage rigoureux qui a motivé la réalisation de **Secure Requirements Studio (SRS)** : un outil interne d'ingénierie des exigences qui mène un entretien structuré avec les parties prenantes (en mode assisté par IA ou en questionnaire classique hors ligne) et transforme progressivement leurs réponses en un Cahier des Charges, une conception applicative, un diagramme de conception MVP et un ensemble d'exigences de cybersécurité — chaque élément généré restant traçable à une réponse effectivement donnée pendant l'entretien, plutôt qu'à un modèle générique recopié d'un projet à l'autre.

1.1 Objectifs du stage

[À COMPLÉTER : Lister les objectifs pédagogiques et professionnels assignés par l'encadrant.]

1.2 Structure du rapport

Ce rapport est structuré comme suit : le chapitre 2 présente l'organisme d'accueil ; le chapitre 3 détaille le contexte et la problématique ; le chapitre 4 couvre l'analyse et la conception du système ; le chapitre 5 décrit la réalisation de Secure Requirements Studio ; le chapitre 6 explique le rôle de l'assistance par intelligence artificielle ; le chapitre 7 détaille les aspects de cybersécurité ; le chapitre 8 présente les tests effectués ; le chapitre 9 discute les résultats obtenus et les limites ; le chapitre 10 conclut le rapport et ouvre sur les perspectives.

Chapitre 2

Présentation de la Fondation OCP

2.1 Présentation générale

[À COMPLÉTER : Présentation officielle de la Fondation OCP : date de création, statut juridique, rattachement au Groupe OCP, gouvernance.]

2.2 Mission et domaines d'intervention

[À COMPLÉTER : Domaines d'intervention de la Fondation (éducation, entrepreneuriat, économie sociale et solidaire, environnement, culture, etc.), sourcés depuis des documents officiels ou le site institutionnel de la Fondation.]

2.3 Contexte de l'Axe Économie Sociale et Solidaire

L'Axe Économie Sociale et Solidaire regroupe les programmes de la Fondation destinés aux coopératives et à leurs bénéficiaires (agriculteurs, femmes, jeunes, personnes en situation de handicap, actifs et inactifs). Ces programmes s'inscrivent également dans une logique de suivi par rapport aux Objectifs de Développement Durable (ODD) et aux indicateurs ESG (Environnement, Social, Gouvernance).

[À COMPLÉTER : Compléter avec le nombre exact de coopératives accompagnées, la répartition géographique, et les chiffres officiels communiqués par la Direction concernée — ne pas extrapoler des chiffres non confirmés.]

Chapitre 3

Contexte et problématique

3.1 Contexte du projet

Le suivi des coopératives et de leurs bénéficiaires au sein de l’Axe Économie Sociale et Solidaire repose aujourd’hui, selon les informations recueillies, en partie sur des fichiers bureautiques (Excel, CSV) et des remontées manuelles, ce qui limite la visibilité consolidée et complique la production de rapports fiables. **[À COMPLÉTER : Confirmer avec la Direction concernée l’état exact de l’existant avant publication du rapport final — ne pas présenter cette description comme un fait officiel non vérifié.]**

3.2 Problématique

Comment concevoir, avant tout développement, une spécification complète, cohérente et sécurisée de la future plateforme de gestion des bénéficiaires — couvrant les besoins fonctionnels, les contraintes techniques et les exigences de cybersécurité — à partir d’un entretien structuré avec les parties prenantes, plutôt que d’un cahier des charges générique ?

3.3 Besoin métier

La Fondation OCP a besoin d’un cadrage rigoureux, traçable et réutilisable des exigences du futur système, garantissant que chaque exigence documentée provient effectivement d’une réponse de partie prenante, d’une règle métier explicite, ou d’une hypothèse clairement identifiée comme telle — et non d’une invention.

3.4 Objectifs du projet

- Fournir un outil d’entretien structuré (IA ou questionnaire classique) dédié à ce projet précis, et non un chatbot généraliste.
- Générer un Cahier des Charges concis (3 à 5 pages) directement dérivé des réponses collectées.
- Produire une conception applicative de haut niveau et un diagramme de conception MVP unique.

- Générer des exigences de cybersécurité proportionnées aux risques effectivement identifiés durant l'entretien.
- Garantir qu'aucune information confidentielle de la Fondation OCP n'est inventée par le système.

3.5 Périmètre

Le périmètre de ce stage couvre la conception et le développement de **Secure Requirements Studio**, l'outil d'ingénierie des exigences. Il ne couvre **pas** le développement de la future Plateforme de Gestion des Bénéficiaires elle-même, qui reste un projet ultérieur distinct, à charge de l'équipe de développement qui reprendra les livrables produits par SRS.

Chapitre 4

Analyse et conception

4.1 Étude de l'existant

[À COMPLÉTER : Décrire les outils/processus existants avant SRS pour le cadrage de projets similaires au sein de la Fondation OCP, si une étude a été menée. Sinon, indiquer explicitement qu'aucune étude formelle de l'existant n'a été réalisée dans le cadre de ce stage.]

4.2 Analyse des besoins

Les besoins ont été structurés autour d'un modèle de spécification centralisé (le *Knowledge Graph*), qui organise les réponses de l'entretien par domaine : parties prenantes, utilisateurs, coopératives, bénéficiaires, exigences fonctionnelles, règles métier, workflows, documents, KPI, ODD, ESG, sécurité, données, intégrations, architecture, infrastructure, notifications, reporting, tests et déploiement.

4.3 Acteurs et rôles

- **Administrateur / consultant SRS** — mène l'entretien de cadrage.
- **Partie prenante interrogée** — représentant métier de la Fondation OCP ou de l'Axe Économie Sociale et Solidaire.
- **Équipe de développement (destinataire des livrables)** — reprend le Cahier des Charges et la conception pour développer la future plateforme.

4.4 Fonctionnalités principales de SRS

- Entretien structuré (Classic Questionnaire hors ligne ou IA adaptative).
- Moteur de Knowledge Graph avec suivi de complétude et de dépendances entre concepts.
- Moteur de génération d'exigences (fonctionnelles, non fonctionnelles, sécurité, techniques).
- Moteur d'analyse de sécurité (score de sécurité, checklist, matrice RBAC, registre des risques, analyse d'impact vie privée).
- Moteur de revue (règles de complétude avant export final).

- Générateur de documents multi-formats (JSON, Markdown, HTML, DOCX, PDF).

4.5 Architecture générale

SRS suit une architecture classique en trois couches :

- **Frontend** — application Next.js (React) assurant l’assistant d’entretien, les tableaux de bord et la consultation des livrables générés.
- **Backend** — API FastAPI (Python) exposant les routes projet, entretien, knowledge graph, exigences, sécurité, revue et documents.
- **Base de données** — PostgreSQL en production (SQLite en environnement de test), via SQLAlchemy asynchrone.

Il n’existe volontairement **aucune couche d’authentification** dans SRS : chaque projet appartient à un contexte interne unique (« Fondation OCP »), l’outil étant destiné à un usage interne restreint pendant la phase de cadrage plutôt qu’à un accès multi-organisation.

4.6 Conception de la solution

La conception distingue explicitement, à chaque étape de la génération documentaire : les informations *confirmées* par une réponse d’entretien, les *hypotheses* formulées par le système, et les points à *confirmer* avec les parties prenantes concernées (par exemple les exigences légales/RGPD ou Loi 09-08, qui sont systématiquement étiquetées comme à valider avec le responsable juridique/DPO).

4.7 Diagramme de conception du MVP

Conformément au principe « un seul diagramme de conception MVP compréhensible par des publics techniques et non techniques », SRS génère dynamiquement un diagramme unique (format Mermaid) représentant les acteurs, le frontend, le backend, les modules effectivement justifiés par les exigences confirmées, la base de données, et les contrôles de sécurité applicables — sans ajout systématique de briques non justifiées (microservices, Kubernetes, files de messages, etc.).

[À COMPLÉTER : Insérer une capture d’écran ou un rendu du diagramme MVP généré pour un projet réel, une fois disponible.]

FIGURE 4.1 – Exemple de diagramme de conception MVP généré par SRS

4.8 Flux principaux

4.8.1 Flux d’entretien

Démarrage de session → question adaptative ou issue du catalogue de concepts → interprétation de la réponse → mise à jour du Knowledge Graph → génération incrémentale des exigences associées → question suivante ou fin d’entretien.

4.8.2 Flux de génération documentaire

Sur demande, le générateur de documents assemble le contenu à partir de l'état courant du Knowledge Graph, des exigences et — si disponible — de l'analyse de sécurité, puis l'exporte dans le format demandé (JSON, Markdown, HTML, DOCX, PDF).

4.8.3 Flux de revue

Avant export final, le moteur de revue vérifie la complétude (parties prenantes, exigences, sécurité, architecture, tests) et distingue les éléments confirmés, supposés et restant à confirmer.

4.9 Choix technologiques

Composant	Technologie
Backend	FastAPI, SQLAlchemy (async), Alembic
Base de données	PostgreSQL (prod), SQLite (tests)
Frontend	Next.js, React, Tailwind CSS
IA (entretien adaptatif)	Google Gemini, via une abstraction de client IA
Export documentaire	python-docx, ReportLab, markdown2
Tests backend	pytest, pytest-asyncio

Chapitre 5

Réalisation de Secure Requirements Studio

5.1 Environnement de développement

- Backend : Python 3.12, FastAPI, Poetry pour la gestion des dépendances.
- Frontend : Node.js, Next.js 15, TypeScript.
- Éditeur : **[À COMPLÉTER : VS Code / autre, selon l'environnement réellement utilisé].**
- Gestion de version : Git. **[À COMPLÉTER : Préciser l'hébergement du dépôt (interne, GitHub, GitLab...) et la politique de branches si applicable.]**

5.2 Architecture de Secure Requirements Studio

Le backend est organisé en couches : **routers** (endpoints HTTP), **services** (logique métier des différents moteurs), **repositories** (accès aux données), **models** (entités SQLAlchemy) et **domain** (connaissances métier statiques, telles que le catalogue de concepts et les référentiels de sécurité).

5.3 Questionnaire guidé

Le *Classic Questionnaire* permet de mener un entretien complet sans dépendre d'un fournisseur d'IA : chaque question est associée à un domaine du Knowledge Graph, avec suivi automatique de la complétude et des dépendances entre concepts (par exemple, une question sur le RBAC régional n'est posée qu'après confirmation du regroupement des coopératives par région).

5.4 Entretien assisté par IA

En mode assisté, l'IA joue le rôle d'un consultant senior en ingénierie des exigences : questions courtes et précises en français, pas de répétition d'une question déjà posée sous la même forme, et restriction stricte au périmètre du projet cadré. Toute question hors périmètre reçoit une réponse de recentrage plutôt qu'une réponse de chatbot généraliste.

5.5 Intégration des fournisseurs IA

[À COMPLÉTER : Décrire précisément, au moment de la rédaction finale, les fournisseurs IA effectivement intégrés et testés (Gemini confirmé dans le code; les autres fournisseurs mentionnés dans les spécifications initiales — OpenAI, Anthropic Claude, Kimi, Ollama — ne doivent être décrits comme livrés que s'ils ont réellement été implémentés et testés au moment de la rédaction).]

5.6 Génération dynamique des exigences

Chaque réponse suffisamment complète déclenche la génération d'exigences typées (fonctionnelle, non fonctionnelle, sécurité, technique, etc.), chacune associée à une clé unique, un identifiant du concept source (`source_concept_key`), une priorité et un statut — garantissant la traçabilité entre une exigence et la réponse qui l'a produite.

5.7 Génération du Cahier des Charges

Le Cahier des Charges est assemblé automatiquement à partir de l'état courant du Knowledge Graph et des exigences générées, avec pour contrainte de rester concis (3 à 5 pages) plutôt que de reproduire un document générique volumineux.

5.8 Génération de la conception

La conception applicative de haut niveau (architecture, acteurs, modules, flux principaux) est dérivée du même modèle de données que le Cahier des Charges, garantissant la cohérence entre les deux documents.

5.9 Génération du diagramme MVP

Voir section 4.7 pour le détail du principe de génération du diagramme unique de conception MVP.

5.10 Génération des documents

Les formats d'export pris en charge sont : JSON (données machine), Markdown (source de vérité textuelle), HTML (dérivé du Markdown), DOCX et PDF (générés directement à partir du modèle de contenu structuré, pour garantir une mise en forme fiable indépendamment de la conversion Markdown).

Chapitre 6

Intelligence artificielle et assistance à l'analyse

6.1 Objectif de l'assistant IA

L'assistant IA n'est pas le produit final de SRS : c'est un intervieweur intelligent dont le rôle est de collecter des informations. Le produit réel est la transformation des réponses de l'entretien en modèle de projet structuré, puis en exigences, en architecture et en documents.

6.2 Fonctionnement de l'entretien intelligent

À chaque tour, l'IA reçoit un contexte structuré (concept courant, historique des échanges, informations déjà connues, informations manquantes) plutôt qu'une simple conversation non structurée, afin de générer une question courte, précise et non redondante.

6.3 Gestion du contexte

Le contexte transmis à l'IA distingue explicitement les informations connues du projet, les hypothèses, et les questions nécessitant une confirmation — évitant que l'IA présente une supposition comme un fait acquis.

6.4 Restrictions du domaine de l'IA

L'IA est strictement cantonnée aux sujets liés au cadrage de la plateforme (besoins, architecture, données, sécurité, déploiement, tests). Toute question hors périmètre reçoit un message de recentrage explicite plutôt qu'une réponse de chatbot généraliste.

6.5 Gestion des fournisseurs IA

Le client IA est conçu derrière une interface (**AIClient**) découplée du reste de l'application, afin de pouvoir ajouter ou remplacer un fournisseur sans modifier le moteur

d'entretien lui-même.

6.6 Fournisseurs IA intégrés

[À COMPLÉTER : Lister ici, avec exactitude, l'état réel de chaque fournisseur au moment de la rédaction finale (implémenté et testé / partiellement implémenté / non implémenté), plutôt que l'ambition initiale du cahier des charges.]

6.7 Sécurité des clés API

Les clés API des fournisseurs IA ne transitent jamais côté client : elles sont lues côté serveur à partir de variables d'environnement, ne sont jamais journalisées, ni incluses dans les documents générés, ni committées dans le dépôt Git.

6.8 Limites et risques liés à l'IA

- Une réponse d'IA reste une interprétation : elle doit toujours être distinguée d'une information officiellement confirmée par une partie prenante.
- En cas d'indisponibilité du fournisseur IA, le Classic Questionnaire reste pleinement fonctionnel afin de ne jamais bloquer un entretien.
- Aucune information confidentielle non confirmée par la Fondation OCP ne doit être présentée comme un fait acquis dans les documents générés.

Chapitre 7

Cybersécurité

La cybersécurité est un axe central du projet, tant dans l’outil SRS lui-même que dans les exigences qu’il génère pour la future plateforme.

7.1 Modèle de menace

Le moteur de sécurité de SRS applique une analyse de type **STRIDE** (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), pondérée selon une logique **DREAD**, sur un catalogue de concepts prioritaires du Knowledge Graph (authentification, autorisation, protection des données, disponibilité, etc.).

7.2 Analyse des risques

Pour chaque risque identifié, un registre structuré est généré, incluant : identifiant, actif concerné, menace, vulnérabilité, impact, probabilité, niveau de risque, mitigation proposée et risque résiduel. Seuls les risques dont le niveau est *élevé* ou *critique* apparaissent dans le registre final, afin de ne pas diluer l’attention sur des risques mineurs.

7.3 Contrôle d’accès

Une matrice RBAC (Role-Based Access Control) est générée à partir des acteurs effectivement identifiés dans les exigences fonctionnelles — elle n’est donc pas fixe, mais reflète les rôles confirmés durant l’entretien (par exemple : administrateur, coordinateur régional, représentant de coopérative).

7.4 Protection des données

Une analyse d’impact relative à la protection des données (PIA) est générée, avec une règle stricte : les exigences légales telles que la Loi 09-08 ou le RGPD ne sont **jamais** présentées comme confirmées par défaut. Elles sont systématiquement étiquetées comme « à confirmer avec le responsable juridique / DPO », sauf confirmation explicite d’une partie prenante durant l’entretien.

7.5 Sécurité des fichiers

[À COMPLÉTER : Détailler les contrôles de sécurité relatifs au téléversement de fichiers (validation de type, taille, scan antivirus) une fois ces exigences confirmées pour un projet concret — ne pas décrire de contrôles qui ne sont pas encore documentés.]

7.6 Journalisation et traçabilité

Chaque exigence générée référence la réponse d'entretien dont elle est issue (`source_concept_key`), fournissant une traçabilité de bout en bout entre une exigence du Cahier des Charges et la réponse de la partie prenante qui l'a motivée.

7.7 Sauvegarde et continuité

[À COMPLÉTER : Documenter la stratégie de sauvegarde et de continuité effectivement retenue pour l'environnement d'hébergement de SRS, une fois le déploiement finalisé.]

7.8 Secure SDLC

Le développement de SRS applique des pratiques de cycle de développement sécurisé de base : séparation stricte de la configuration sensible (clés API) via variables d'environnement, absence de journalisation des secrets, et couverture par tests automatisés des principaux flux applicatifs (voir chapitre 8).

Chapitre 8

Tests

8.1 Tests fonctionnels

Le backend est couvert par une suite de tests automatisés (`pytest`, `pytest-asyncio`) exécutée contre une base SQLite en mémoire, couvrant notamment : la création et le cycle de vie des projets, l'amorçage automatique du Knowledge Graph, le déroulement complet d'un entretien, la génération des exigences, l'analyse de sécurité, le moteur de revue et la génération documentaire multi-formats.

[À COMPLÉTER : Indiquer le nombre exact de tests et le taux de couverture au moment de la rédaction finale du rapport (à régénérer via `pytest-cov` juste avant la remise).]

8.2 Tests d'intégration

Les tests d'API exercent les endpoints HTTP de bout en bout via un client de test asynchrone, garantissant que les routes, les schémas Pydantic et les services métier restent cohérents entre eux.

8.3 Tests de sécurité

Le moteur de sécurité est testé sur des cas explicites : absence de fabrication d'une base légale de traitement des données non confirmée, cohérence entre les acteurs déclarés dans les exigences et la matrice RBAC générée, et non-duplication d'une analyse de sécurité déjà exécutée pour un même projet.

8.4 Tests de génération documentaire

Le générateur de documents est testé pour garantir qu'un projet fraîchement créé (sans réponse d'entretien) ne produit **aucune donnée fabriquée** : dictionnaire de données vide, absence d'analyse de sécurité tant qu'elle n'a pas été exécutée, et mention explicite des sections volontairement non générées (diagrammes BPMN, maquettes visuelles).

8.5 Tests de l'assistant IA

[À COMPLÉTER : Décrire les tests spécifiques à l'assistant IA (longueur des questions, respect du périmètre du projet, gestion des erreurs de connexion au fournisseur) une fois formalisés et automatisés.]

Chapitre 9

Résultats

9.1 Fonctionnalités réalisées

Au moment de la rédaction, Secure Requirements Studio permet :

- de mener un entretien de cadrage complet, en mode Classic Questionnaire ou assisté par IA (Gemini) ;
- de générer dynamiquement des exigences fonctionnelles, non fonctionnelles, de sécurité et techniques, traçables aux réponses de l'entretien ;
- de produire une analyse de sécurité (score, checklist, matrice RBAC, registre des risques, analyse d'impact vie privée) ;
- d'exécuter une revue de complétude avant export ;
- de générer un Cahier des Charges et une conception applicative avec un diagramme MVP unique, exportables en JSON, Markdown, HTML, DOCX et PDF.

9.2 Résultats obtenus

[À COMPLÉTER : Compléter avec des résultats concrets une fois un ou plusieurs entretiens réels menés avec des parties prenantes de la Fondation OCP — ne pas anticiper de résultats métier qui n'ont pas encore été produits.]

9.3 Limites

- SRS produit une *conception et une spécification*, pas une implémentation de la future Plateforme de Gestion des Bénéficiaires elle-même.
- L'intégration de certains fournisseurs IA envisagés initialement (OpenAI, Anthropic Claude, Kimi, Ollama) peut être partielle ou absente selon l'état réel du code au moment de la rédaction — voir chapitre 6.
- L'outil ne dispose volontairement d'aucune authentification, ce qui limite son usage à un contexte interne restreint.
- **[À COMPLÉTER : Ajouter toute autre limite constatée en conditions réelles d'usage.]**

9.4 Perspectives

- Étendre la couverture des fournisseurs IA supportés.
- Enrichir le catalogue de concepts du Knowledge Graph au fur et à mesure des entretiens réels menés avec les parties prenantes de l’Axe Économie Sociale et Solidaire.
- Transmettre les livrables générés (Cahier des Charges, conception, diagramme MVP, exigences de sécurité) à l’équipe de développement chargée de la future Plateforme de Gestion des Bénéficiaires.

Chapitre 10

Conclusion générale

Ce stage a porté sur la conception et le développement de **Secure Requirements Studio**, un outil interne d'ingénierie des exigences destiné à cadrer le futur projet de Plateforme de Gestion des Bénéficiaires de l'Axe Économie Sociale et Solidaire de la Fondation OCP, en intégrant dès l'origine des exigences de cybersécurité.

Le travail réalisé a permis de mettre en place un entretien structuré — assisté par IA ou en questionnaire classique hors ligne — capable de transformer les réponses des parties prenantes en un modèle de projet structuré (Knowledge Graph), puis en exigences traçables, en une analyse de sécurité proportionnée aux risques réels, et en un Cahier des Charges concis accompagné d'une conception applicative et d'un diagramme de conception MVP unique.

Au-delà de l'aspect technique, ce stage a été l'occasion de travailler un principe directeur exigeant : ne jamais présenter une supposition ou une inférence de l'IA comme un fait acquis de la Fondation OCP, et distinguer systématiquement ce qui est confirmé, supposé, ou à confirmer. Ce principe a structuré aussi bien la conception du moteur de génération d'exigences que celle du moteur de sécurité.

[À COMPLÉTER : Compléter cette conclusion avec un bilan personnel du stage (apprentissages techniques et humains, difficultés rencontrées et comment elles ont été surmontées) une fois le stage arrivé à son terme.]

Les livrables produits par Secure Requirements Studio — Cahier des Charges, conception applicative, diagramme MVP et exigences de cybersécurité — constituent la base sur laquelle l'équipe de développement pourra s'appuyer pour concevoir, dans un second temps, la Plateforme de Gestion des Bénéficiaires elle-même.

Annexe A

Glossaire

Cahier des Charges Document synthétique (3 à 5 pages dans SRS) décrivant les besoins fonctionnels et non fonctionnels essentiels d'un projet.

Knowledge Graph Modèle interne de SRS structurant les réponses de l'entretien par domaine et par dépendances entre concepts.

RBAC *Role-Based Access Control* — contrôle d'accès basé sur les rôles.

STRIDE Méthode de modélisation des menaces (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege).

DREAD Méthode de pondération des risques (Damage, Reproducibility, Exploitability, Affected users, Discoverability).

PIA *Privacy Impact Assessment* — analyse d'impact relative à la protection des données.

MVP *Minimum Viable Product* — version minimale mais fonctionnelle d'un produit.

ODD Objectifs de Développement Durable (Nations Unies).

ESG Environnement, Social, Gouvernance — cadre d'indicateurs de durabilité.

SRS Secure Requirements Studio — l'outil objet de ce rapport.

Annexe B

Bibliographie et Webographie

Les références utilisées sont listées dans le fichier `references.bib` et citées au fil du texte selon le style bibliographique standard. **[À COMPLÉTER : Ajouter les références effectivement consultées durant le stage (documentation FastAPI, Next.js, OWASP ASVS, OWASP Top 10, documentation Fondation OCP, etc.) au fur et à mesure de la rédaction.]**

- OWASP Foundation, *OWASP Top 10*, <https://owasp.org/www-project-top-ten/>.
- OWASP Foundation, *OWASP Application Security Verification Standard (ASVS)*, <https://owasp.org/www-project-application-security-verification-standard/>.
- FastAPI, documentation officielle, <https://fastapi.tiangolo.com/>.
- Next.js, documentation officielle, <https://nextjs.org/docs>.
- **[À COMPLÉTER : Site institutionnel de la Fondation OCP (page Économie Sociale et Solidaire) — ajouter l'URL exacte consultée.]**

Annexe C

Guide d'installation

C.1 Prérequis

- Python 3.12 et Poetry (backend).
- Node.js (frontend Next.js).
- Docker et Docker Compose (base de données PostgreSQL et Redis).

C.2 Démarrage rapide

Le dépôt fournit des scripts de démarrage prêts à l'emploi :

- `run.bat` / `run.ps1` — Windows.
- `run` / `runlinux` — Linux / macOS / WSL.

Ces scripts vérifient la présence d'un fichier `backend/.env` (le créent à partir de `.env.example` si absent), démarrent les services d'infrastructure via `docker-compose.yml` (PostgreSQL, Redis), lancent le backend FastAPI puis le frontend Next.js, et affichent les URLs locales.

C.3 Configuration

Les variables d'environnement sensibles (clés API des fournisseurs IA, chaîne de connexion à la base de données) sont définies dans `backend/.env`, jamais committées dans le dépôt Git — seul `backend/.env.example` y figure, sans valeur réelle.

C.4 Lancement manuel (sans script)

```
# Infrastructure (Postgres + Redis)
docker compose up -d

# Backend
cd backend
poetry install
poetry run uvicorn app.main:app --reload

# Frontend (dans un autre terminal)
```

```
cd frontend  
npm install  
npm run dev
```

C.5 Tests

```
# Backend  
cd backend  
poetry run pytest  
  
# Frontend  
cd frontend  
npx tsc --noEmit  
npm run build
```

[À COMPLÉTER : Mettre à jour cette annexe si la procédure de déploiement change avant la remise finale du rapport (voir également DEPLOYMENT.md à la racine du dépôt, si présent).]

Annexe D

Annexes techniques

D.1 Arborescence simplifiée du backend

backend/app/	
routers/	endpoints HTTP (projects, interview, knowledge-
graph,	requirements, security, review, documents, ai-
	settings)
services/	logique métier (interview, knowledge_graph,
requirement,	security_engine, review_engine, document_generator
	,
	document_rendering, llm_client, ai_settings)
repositories/	accès aux données (une classe par entité)
models/	entités SQLAlchemy
schemas/	schémas Pydantic (validation / sérialisation API)
domain/	connaissances métier statiques (catalogue de
concepts,	types d'exigences, référentiels de sécurité,
	savoir FOCP)

D.2 Exemple d'exigence générée

Extrait illustratif de la structure d'une exigence générée par SRS (les valeurs sont indicatives) :

```
{
  "requirement_key": "SEC-VAL-001",
  "requirement_type": "security",
  "title": "Validation des donnees par un role autorise",
  "priority": "haute",
  "source_concept_key": "workflows.monthly_validation",
  "acceptance_criteria": [
    "Seuls les roles autorises peuvent valider une soumission mensuelle."
  ]
}
```

D.3 Exemple de diagramme MVP (Mermaid)

Extrait illustratif du diagramme de conception MVP généré dynamiquement (structure simplifiée — le contenu réel dépend des exigences confirmées pour chaque projet) :

```
flowchart TD
    AO["Administrateur"] --> FE["Frontend"]
    A1["Coordinateur regional"] --> FE
    FE --> API["Backend / API"]
    API --> MO["Exigences fonctionnelles"]
    MO --> DB["(Base de donnees)"]
    API --> SEC["Controles de securite (RBAC, audit, chiffrement)"]
```

[À COMPLÉTER : Remplacer ces extraits par des exemples réels générés pour le projet Fondation OCP, une fois un entretien complet mené et un cahier des charges effectivement exporté.]